

A profitable trading algorithm for cryptocurrencies using a Neural Network model

Mimmo Parente^a, Luca Rizzuti^a, Mario Trerotola^a

^a *Dipartimento di Scienze Aziendali - Management and Innovation Systems,
Università degli Studi di Salerno, Italy*

Abstract

Algorithmic trading enables the execution of orders using a set of rules determined by a computer program. Orders are submitted based on an asset's expected price in the future, an approach well suited for high-volatility markets, such as those trading in cryptocurrencies. The goal of this study is to find a reliable and profitable model to predict the future direction of a crypto asset's price based on publicly available historical data. We first develop a novel labeling scheme and map this problem into a Machine Learning classification problem. The model is then validated on three major cryptocurrencies through an extensive backtest over both a bull and a bear market. Finally, the contribution of each feature to the classification output is analyzed.

Keywords: Cryptocurrencies, Machine Learning, Neural Network, Price Prediction, Algorithmic Trading, Explainable AI, Backtesting, Shapley Values

1. Introduction

A financial trading system on public market exchanges comprises a set of rules and tools that help the trader agent make the best decisions during the investment phase. These rules and tools are trading algorithms applied to data relating to one or more financial assets, in order to identify and exploit profit opportunities. Multiple data sources can be used to make trading decisions, and one of the most widely publicized projects using such techniques

Email addresses: parente@unisa.it (Mimmo Parente), lrizzuti@unisa.it (Luca Rizzuti), mariotrerotola@gmail.com (Mario Trerotola)

Preprint submitted to Expert Systems With Applications

June 6, 2023

in financial applications is *Standard & Poor's Neural Fair Value 25* portfolio. This uses an artificial neural network to select 25 stocks on a weekly basis from a total of 3000 stocks, with the aim of outperforming the market by calculating a stock's weekly fair value based on fundamental analysis. In the field of securities trading, the utility of complex models such as Neural Networks (NN), Support Vector Machines (SVM) and hybrid models has been extensively studied and promising results have been obtained [?]. However, information regarding the incorporation of such methods into trading floor operations tends to remain hidden to the public, for commercial proprietary reasons [?]. Hence, there is an ample of room to explore automated trading using modern machine learning (ML) approaches.

In this paper, we are interested in cryptocurrency trading. Cryptocurrency-related assets have seen a significant increase in market acceptance and have developed rapidly in recent years. As a result, many hedge funds and investors are beginning to include this type of asset in their financial portfolios, which has had a significant impact on the overall market and has generated considerable interest in trading algorithms for cryptocurrencies. According to [?], 85% of the scientific papers on the algorithmic trading of cryptocurrency-related assets have been published since 2018. While strategies applied to traditional financial markets can be adapted to the cryptocurrency market, they have certain unique characteristics that require new research efforts from the scientific community. Standard trading approaches are based on *fundamental* and *technical* analysis. The fundamental approach aims to determine whether an asset is currently traded at its fair market value using financial metrics and by conducting thorough research of the asset's business-related information. Technical analysis, on the other hand, analyzes price and volume series in order to predict an asset's future value, although this is founded on the assumption that future market prices can, in fact, be predicted.

According to the Efficient Market Hypothesis, [?], market agents are rational and new information is immediately reflected in the price, whereas the Adaptive Market Hypothesis [?] suggests that investors may behave irrationally in response to market volatility, thereby creating buying opportunities. This can be ascribed to human behavior, such as loss aversion, overconfidence and overreaction, which can increase market volatility in certain circumstances. The inefficiency of the major cryptocurrencies is investigated by [?] concluding that "results indicate that all these cryptocurrencies are inefficient markets". This makes the cryptocurrency market an ideal scenario for applying technical analysis using automated agents. Another peculiar-

ity of this scenario is that digital exchanges, and especially cryptocurrency exchanges, have lower fees than traditional brokers, which can make a significant difference in the high volume and frequencies of trades typical in this market. At present, a transaction fee on cryptocurrency exchanges can be as low as 0.1% per trade (at the time of writing, such fees are reduced to 0% on specific currency pairs). Additionally, most exchanges provide free trading API, thus further reducing barriers to entry for algorithmic trading.

1.1. Our contribution

We have developed a trading algorithm based on a Multi-Layer Perceptron (MLP) as a classifier with three classes, Buy, Hold and Sell. We designed a complete usual analysis pipeline: first we gathered price and volume time-series from a popular cryptocurrency exchange, then preprocessed them by feature extraction and labeling, and finally trained and tested the MLP. One of the peculiarities of our work is the dataset labeling algorithm, which is based on two thresholds to intercept significant market movements and two temporal windows, one in the past and one in the future for the forecasting of price trends.

A true trading strategy must be profitable in every phase of the market cycle, both in the bull market characterizing the past 10 years and in the bear/highly volatile market such as the one at the time of writing. Our findings show that a perfectly validated model with good performance standard metrics might perform poorly when used in a real simulation of a highly volatile market. To further validate our model, we set a simulation on a real scenario based on long term off-label historical data for different currencies (backtest phase).

We built a massive dataset of hundreds of cryptocurrencies spanning several years at a 4-hour time resolution. The strategy was then evaluated over two different time intervals in order to assess its behavior on a long-term and a short-term real case scenario, which yielded very positive results in terms of Return Of Investment (ROI). We then concluded the study with an analysis of the feature importance.

The Python source code developed for this research, datasets and output of the analysis can be found at [?].

This paper is organized as follows: in Section 2 we review the relevant literature, analyzing it according to the different types of data sources used to predict cryptocurrency market trends. In Section 3 we show the preprocessing and feature extraction pipeline and the algorithm for data labeling.

In Section 4 we describe the design of the MLP neural networks and rank these models according to the accuracy achieved in the testing phase. In Section 5 we report the results of the simulation. In Section 6 we describe the feature importance analysis conducted. Finally in Section 7 we give some conclusions and hints on future directions to be explored.

2. Related works

Forecasting the asset price or its direction calls for significant work on data integration and feature extraction from raw market data and/or other sources.

In [?] , [?] , and [?] sentiment analysis is employed on Twitter data feeds to predict the prices of major cryptocurrencies using standard ML models (Random Forest Regressor, SVM, MLP). [?] uses messages from cryptocurrency-related web forums as primary data sources employing a probabilistic model based on Averaged One-Dependence Estimators (AODE).

In [?] data are taken from blockchain, coin exchanges and Google Trends to predict Bitcoin price. The authors derived new features from the size of transactions and employed a proprietary model based on wavelet decomposition for preprocessing data and a Causal Multi-Head Attention Temporal Convolutional Network (WT-CATCN) for the modeling layer. Similarly, [?] encodes transactions found in the Bitcoin blockchain into a graph, searches for recurrent patterns, and correlates them with price changes using standard ML models (MLP, SVM). The authors in [?] find that the Ethereum price is strongly positively correlated with various user activities on its blockchain, but they also identify an inverse correlation with the crude oil trend, probably due to the rise in energy costs.

Some research methods borrowed from traditional market methodologies seek correlations between cryptocurrencies and macroeconomic and/or financial indicators. [?] evaluates macro and financial indexes to model the volatility of prices for five major cryptocurrencies by utilizing a GARCH-MIDAS framework [?] . Similarly, [?] predicts the daily Bitcoin price by using data from other commodities and indexes, transforming them into the time-frequency domain using wavelet decomposition, and feeding the data into a Long Short-Term Memory (LSTM) network. [?] also uses macroeconomic indexes and blockchain information to forecast the Ethereum price.

In addition to external data sources, some papers exploit the information in the order book. The order book contains the *limit orders* on both Buy and Sell sides, registered by the market participants. The orders placed by big market players, are often used as a source of information to predict the near-term direction of the asset's prices. [?] and [?] use Convolutional NN and SVM models respectively. Similarly, [?] uses order book data to complement the price and volume information in order to extract features and predict prices.

Seasonality is another area of research borrowed from standard assets whose effects are evaluated on cryptocurrencies, see [?] and [?].

Many papers rely on prices and volumes time series, basing the feature extraction on technical analysis or using techniques for time series forecasting/regression. In [?] the daily prices of the 3 topmost cryptocurrencies are used to forecast the next day's prices with LSTM and Generalized Regression Neural Network. Similarly, [?] uses different neural network architectures to assess the predictability of trend direction in a minute time frame and finds that the hybrid LSTM with the Convolutional layer between the input and the Neural Network yields the best results.

Some papers seek to validate their methodology by carrying out a simulation on historical data, known as *backtesting*. In [?], the simulation is conducted on a trading session over a 12-week time interval, far outperforming the baseline and giving an excellent 30% ROI. The authors even find that the random baseline strategy of investment has an 11% ROI, although their strategy was tested on a short and relatively quiet period.

The labeling of datasets for supervised learning is an open problem that is, broadly speaking, tackled using variations of two main ideas: price regression and price direction forecasting. In the former the differences lie in the size of the time frame used to assign a label (minutes, hours, days, etc.) and in the latter the difference is whether 2 or 3 labels are used, for Buy/Sell or Buy/Hold/Sell. In this paper we use three labels for the classification (as only [?] and [?] do, among all those papers cited above).

3. The dataset

In this section we describe the dataset used to apply our supervised ML algorithm. It consists of raw data gathered from the price and volume data time series of a popular crypto-currency exchange. Then we show how the features were extracted from the raw data by computing candlestick patterns

and financial indicators, and by moving average crossovers and temporal data. Finally, we design an algorithm to label the observations, parameterized in such a way that it can be used to maximize classification accuracy and the profitability of a given trading strategy.

3.1. Data collection

Data were downloaded from a popular crypto exchange platform exposing web based APIs and were collected in the usual *HOLC + volume* format, which encodes price variation in a given time frame in terms of *High*, *Open*, *Low*, *Close prices* and *Volume* exchanged during the given time frame. As usual *HOLC* is represented by “candlesticks” in the asset’s price charts. We collected data for 402 different crypto assets and set the time frame length at 4 hours. The assets were all those available on the platform from August 17, 2017 to December 4, 2022. For those assets not available at the starting date, data were collected as soon as they were listed on the exchange. All the cryptocurrencies are those paired with the stable coin United States Dollar Tether (USDT). After the feature extraction phase, we obtained a dataset of 1.5 million samples with the distribution shown in Figure 1.

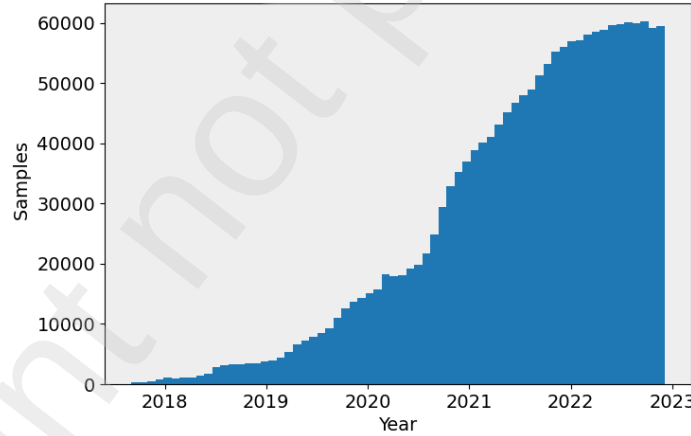


Figure 1: Cryptocurrency samples distribution over time

3.2. Feature extraction

We used the *HOLC* prices and the volumes to compute some technical analysis indicators and patterns to synthesize past price behaviors. Specifi-

cally, we used candlestick patterns, financial indicators, and temporal data to spot future price direction.

Candlestick patterns. Candlestick charts are typically used to visualize price fluctuations. A series of candlesticks may form a pattern and constitute a technical analysis tool that can suggest future price movements on the basis of past price behavior. By definition, the patterns are independent of the size of the time frame [?]. Our study considers 23 of the most popular candlestick patterns, both single and multiple candles, in their bullish and bearish version, e.g. Three Black Crows, Doji, Engulfing, Hammer. In this paper we use all the patterns mentioned in [?].

Technical indicators, known as oscillators, can assume values in a zero-centered interval and can be used as-is as features in ML applications. We use 6 of the most common technical indicators: Bollinger bands, ULTOSC, RSI, Close price percentage variation, Z-Score and volume Z-Score. Moreover, we also consider the Exponential Moving Average crossover so as to take into account the trends and trend reversals. See [?] and [?] for more on technical indicators for financial markets.

- Bollinger bands. These are used to check whether prices are high or low on a relative basis. Given n periods, the price calculated using the n -period moving average is used as a reference price. Two lines are plotted above and below one standard deviation away from the reference price. The standard deviation of the price in the previous n -periods is used as a measure of volatility.
- RSI. The Relative Strength Index is a momentum indicator that measures the magnitude of recent price changes in order to evaluate overbought and oversold conditions. A lower RSI value indicates that the asset is oversold, while a higher value means that the asset is overbought.
- ULTOSC. The Ultimate Oscillator uses the values of three different moving averages with multiple time periods (or cycles), to identify overbought and oversold conditions in the market, thus improving the accuracy of the signals generated by the indicator.
- Close price percentage variation. This measures the percentage difference between the current price and the previous close price.

- **Z-Score.** This uses the z-score of the close price in a given number of time frames. In our implementation we used 30 close price past samples to compute the Z-Score of the actual time frame.
- **Volume Z-score:** In order to convert transaction volumes for comparing those of different cryptocurrencies, we used z-score normalization.
- **EMA crossovers.** The Exponential Moving Average crossovers are an established source of information for trend following and inversion. We considered 4 EMA crossovers based on 1 and 20, 20 and 50, 50 and 100, and 1 and 50 periods.

Temporal information. In Section 2 we noted that temporal information adds a statistical edge to price direction prediction. Thus, we have also added three other features based on time. Every single OHLC sample has an associated timestamp, used to extract the month of the year, the day of the week and the number of samples in the day (six a day). Summarizing, the overall feature vector has 36 entries: 23 candlestick patterns, 6 financial indicators, 4 EMA crossovers, and 3 temporal features.

3.3. Labeling Algorithm

The analysis we conducted uses a *supervised* ML model, thus each sample of the dataset has to be labeled for the model’s training phase. First let us define the temporal *windows* and the *return* for a single trade operation.

Definition 1. Given a time frame at time t and an crypto currency c , a **Forward Window** of size k , $\mathcal{FW}_{t,c}(k)$, is the sequence of time frames $t, t+1, \dots, t+k-1$ and a **Backward Window** of size k , $\mathcal{BW}_{t,c}(k)$, is the sequence of time frames $t-k+1, t-k+2, \dots, t$.

In the following we drop the subscripts t and c when the context is clear. Opening a position in the market at the open price of the time frame at time t , $Open_t$, and closing the position at the close price of the time frame $t+k$, $Close_{t+k}$, yields a revenue or a loss, formally defined as follows.

Definition 2. The **return** of the trade of a crypto currency c , opened at time t and closed at the time frame $t+k$, is given by:

$$\mathcal{R}_{t,c}(k) = \frac{(1-f) \cdot Close_{t+k} - (1+f) \cdot Open_t}{Open_t}$$

where f is the fee applied by the exchange for each trade operation¹.

Here too we drop the subscripts t and c and retain only the duration k of the opened position, when clear from the context.

The labeling algorithm has two parameters, α and β , which are used to set the thresholds of the return values $\mathcal{R}(k)$, where k is the size of the *Forward* window. The former is used to establish a low value below which it is not convenient to place a trading order and the latter is a high value above which we consider a price variation not influenced by the technical framework but by exogenous forces. In this extreme scenario we likewise do not place orders. In summary, in order to trigger a Buy, Sell or Hold signal, given $Open_t$ we want to predict the price variation at the end of the Forward window $Close_{t+k}$. Figure 2 depicts the behavior of the labeling algorithm as a function of the α and β parameters.

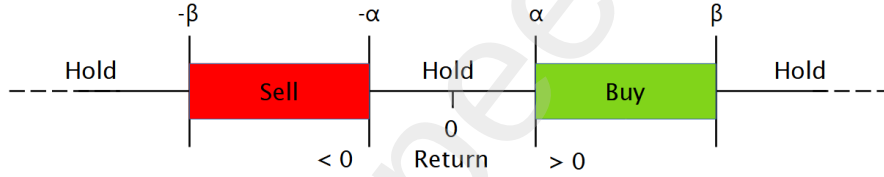


Figure 2: Visual representation of label boundary based on the α and β parameters

Algorithm 1 receives in input the array $closePs$ of the close prices of all the time frames, the sizes of the Backward and Forward windows, $backW$ and $forW$, and the values of α and β . In line 2, the prices of the backward windows are smoothed by computing the Exponential Moving Average (EMA) so as to remove any random fluctuations or noise resulting from unpredictable events that may have influenced the market. The EMA is computed backwards on previous $backW$ close prices and its value is taken as the reference of the open price for the next time frame.

Given a forward window $\mathcal{FW}_{t,c}(forW)$, the label at time frame t is saved at line 14. The return is computed at line 4 and checked to determine whether it is within $]\alpha, \beta[$, in absolute value, at line 5. The label is set to Buy at line 7 if the return is positive, and it is set to Sell in line 9 if negative. In the event of a return that is not significantly varied, when it falls within $[-\alpha, +\alpha]$, or

¹The fee is given here as a percentage of the investment and, for simplicity's sake, is assumed to be equal for both a buy and a sell operation.

in case of a very high magnitude variation (the absolute value of variation greater than β), then the label is set to **Hold**, at line 12.

Algorithm 1 Labeling algorithm

Input: $ClosePs[]$: close prices time series

$backW$: backward window size

$forW$: forward window size

α : low threshold

β : high threshold

Output: $Labels$: Array of Buy, Hold and Sell labels for close price

```

1: procedure COMPUTELABELS( $closePs[], backW, forW, \alpha, \beta$ )
2:    $openPs[] \leftarrow ema(closePs, backW)$ 
3:   for  $t \leftarrow 1$  to  $len(closePs)$  do
4:      $return \leftarrow (closePs[t + forW] - openPs[t]) / openPs[t]$ 
5:     if  $\alpha < abs(return) < \beta$  then
6:       if  $return > 0$  then
7:          $label \leftarrow Buy$ 
8:       else
9:          $label \leftarrow Sell$ 
10:      end if
11:    else
12:       $label \leftarrow Hold$ 
13:    end if
14:     $Labels[t] \leftarrow label$ 
15:  end for
16:  return  $Labels$ 
17: end procedure

```

Figure 3 depicts a price chart. The rectangles represent the windows around a price sample, marked with an arrow. The *Backward* window is of size 5 and the *Forward* window is of size 2. The 5EMA of the close prices in the backward window is the starting point of a trend, shown as a dotted line, giving the price direction to compute the label of the current price sample.

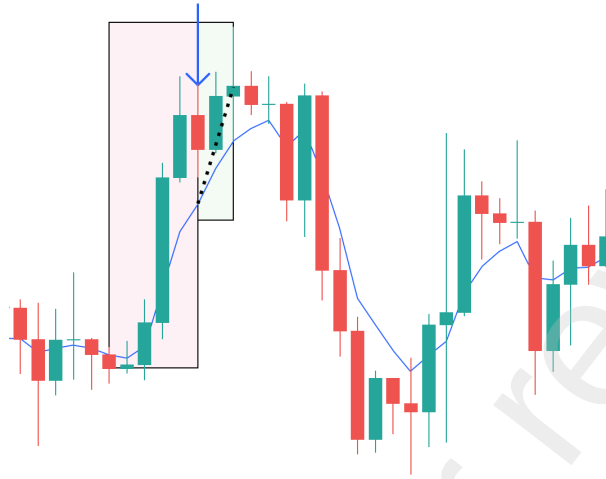


Figure 3: A price chart with backward and forward windows of sizes 5 and 2, respectively, with a positive price variation.

4. MLP Models, training and testing

In this section we introduce the classification model and the methodology applied for the training and the testing phase. We adopted the MLP model with four layers as a classifier, trained on the dataset labeled with Algorithm 1.

4.1. Multi-Layer Perceptron

The MLP consists of an input layer, two hidden densely connected layers and a categorical output layer with three nodes. When defining the MLP architecture in terms of the number of layers and neurons, we followed two guidelines: the universal approximation theorem [?] and the principle that an MLP should have the least number of neurons to generalize well [?]. The universal approximation theorem ensures that a feedforward neural network with one input layer, one hidden layer and one output layer can approximate any function of the input with any desired degree of accuracy, provided that a sufficient number of hidden units are available.

Using a bottom-up approach, see e.g. [?], we began with a small number of neurons on each hidden layer and compared the accuracy on the train and test set. On undersized networks, these two numbers are comparable up to the second digit after the decimal point. As we added neurons to each hidden layer, the network began to overfit on the training set. We

selected the minimum number of neurons that exhibited slight overfitting and provided the highest accuracy on the test set. This approach allowed us to avoid managing overfitting with dropout layers or l1/l2 penalty terms on the weights of the network.

Specifically, the input layer has 128 nodes, the two hidden layers have 64 and 32 nodes, respectively, all with the LeakyReLU activation function. Finally, the output layer has three nodes with the Softmax activation function, which ensures a probabilistic output for the three classes.

4.2. Window sizes, α and β parameters

Forward and *Backward* windows, and α and β , described in Section 3.3, are the parameters and the thresholds used to label the dataset. Here we describe how these values are computed. The thresholds α and β emerge from a statistical analysis of the open-close percentage change in prices for the whole dataset. The value of α is set to the 85-th percentile and the value of β is set to the 99.7-th percentile. The absolute values of the thresholds are thus $\alpha = 0.038$ and $\beta = 0.24$. The β marking the outlier boundary, see Figure 2, is incremented by 10% each time the size of the *Forward* window is increased. Thus, for windows of size 2, $\beta = 0.24 + 0.024$, for size 3, $\beta = 0.24 + 2 * 0.024$ and so on.

This choice of α leads to an unbalanced dataset towards the *Hold* class, which represented approximately the 70%, resulting in poor classification results. To overcome this, we used a random undersampling on the majority class to balance the dataset, see e.g. [?].

Finally, to obtain the best combination of *Forward* and *Backward* window sizes, a grid search was conducted using the sizes from 1 to 5, hence considering 25 labeling schemes. The training and testing dataset was built with data on currencies from August 17, 2017 to December 31, 2021, and from April 1, 2022 to December 4, 2022 with the 25 different labeling schemes (the first three months of the year 2022 were later used to select the best model by profitability as described in Section 5).

For each combination of *Forward* and *Backward* window sizes, we trained and tested the model three times, with a proportion of 70%-30%, using different random seeds and averaging the results. We then selected the top five accuracy value combinations of *Forward* and *Backward* windows as reported in Table 1. The different window lengths may lead to considerable variations in the size of the samples reported in the last column (due to the rebalancing of the majority class).

BackW	ForW	Acc	Buy		Hold		Sell		Samples
			Pre	Rec	Pre	Rec	Pre	Rec	
5	1	0.72	0.71	0.75	0.67	0.59	0.76	0.81	463k
4	1	0.70	0.69	0.72	0.65	0.60	0.74	0.77	422k
3	1	0.67	0.65	0.67	0.65	0.58	0.69	0.74	379k
5	2	0.66	0.66	0.66	0.61	0.57	0.70	0.75	610k
4	2	0.64	0.64	0.64	0.61	0.55	0.68	0.73	582k

Table 1: Top 5 models ranked by accuracy (third column) and the corresponding combination of Backward and Forward window sizes (first and second column).

5. Backtesting

In Section 4 we described the design and the training of an MLP and selected the top five combinations, in order of accuracy, of *Forward* and *Backward* window sizes used to label the dataset (see Table 1). Then we computed the profit on a simple trading strategy using the same data on these 5 models and we obtained a different ranking of the combination of window sizes. Hence, profitability is an evident and irrefutable metric when assessing algorithmic trading strategies (see e.g. [?]) and we were, therefore, induced to further investigate this situation. *Backtesting* is a standard method for evaluating ex-post the performance of an algorithmic trading strategy which uses profitability as a measure of the goodness of the model adopted. To simulate a real trading scenario, it uses historical data and does not open multiple long or short positions on the same asset simultaneously. At the end of the simulation period, all positions are closed and the profit is computed.

The model to be used in backtesting was selected by means of a specially designed simulation algorithm implementing a standard strategy (see e.g. [?]): the algorithm scans the list of OHLC prices searching for the first **Buy**, which is the signal to enter the market, whereupon it places an order. The algorithm then looks for the next **Sell** label and the position is closed. The earning/loss is capitalized and the strategy restarts, thus obtaining compound interest (an implicit safeguard stop loss has also been implemented). In our accounts we included the commission fees of 0.1% applied by some market brokers at the time of writing. The strategy used the data from the first three months of 2022, (not used in the training/testing phases, see Section 4.2). The period was chosen since the Bitcoin and Ethereum assets were following a downward trend (final price < initial price). The model

with the windows sizes (5, 2), the fourth in Table 1, obtained the greatest return, averaged over all the assets, and it was thus selected for the final backtesting. This model was then trained on the data from August 17, 2017 to December 4, 2022 for all the currencies apart from the data relative to *Bitcoin*, *Ethereum* and *Algorand* which were later used for backtesting.

The result of the test is reported in Table 2. The performance metrics are comparable to those in Table 1, even though the data for *Bitcoin*, *Ethereum* and *Algorand* are missing. We interpret this as an indicator of *good model generalization* on generic technical patterns not bound to specific coins.

	Acc	Buy			Hold			Sell			Samples
		Pre	Rec	F1	Pre	Rec	F1	Pre	Rec	F1	
Test	0.66	0.65	0.68	0.67	0.65	0.52	0.57	0.70	0.78	0.73	201k
Train	0.68	0.66	0.70	0.68	0.66	0.53	0.58	0.71	0.80	0.75	469k
Dummy	0.33	0.32	0.32	0.32	0.32	0.32	0.32	0.36	0.36	0.36	201k

Table 2: Performance and metrics for the model selected for the backtest

The size of the dataset is different because of the differences in the composition of the datasets: those in Table 1 had all the coins except for 3 months of data for everyone, in Table 2 there are all coins except those 3 selected for the backtest.

Once the model had been selected, trained and tested, we extracted the features to finalize the backtesting for the above three currencies over 5 years and 4 months for *Bitcoin* and *Ethereum* and 3 years and 6 months for *Algorand*, and then queried the model. Trading strategies can be described unambiguously using regular expressions on an alphabet composed of three symbols: **Hold**, **Buy**, **Sell**. In this way the trading strategy described above can be written as:

$$(\text{Buy}[\text{Buy}|\text{Hold}]*\text{Sell})^+ \quad (1)$$

where $*$ and $+$ are the transitive closures, see [?] for more on regular expressions.

To assess the quality of the equity curves, we compared our model with a baseline dummy classifier that randomly responds with the same distribution of labels in the unbalanced original dataset, where the **Hold**, **Buy**, **Sell** labels are approximately in the 70,15,15 proportion, respectively. The overall return R for n trades on the coin c , is then computed as follows:

$$R = \prod_{i=1}^n (1 + \mathcal{R}_c^i(2)) \quad (2)$$

where: n is the number of trades executed and $\mathcal{R}_c^i(2)$ is the return of a single trade computed according to Definition 2 on a *Forward* window of size 2.

Table 3 reports the ROIs of all the periods for every single backtest, both for the MLP and the Dummy models, computed as:

$$ROI = \frac{FinalValueofInvestment - InitalValueofInvestment}{InitalValueofInvestment} \quad (3)$$

Period	Coin	ROI MLP	ROI Dummy
long	ALGO	6.69	-0.99
short	ALGO	0.64	-0.42
long	BTC	29.51	-0.77
short	BTC	0.26	-0.44
long	ETH	196.27	-0.91
short	ETH	0.81	-0.41

Table 3: MLP and dummy model backtesting performance

The ROIs are largely positive: the peak is reached for *Ethereum* in the long period with a ROI of 196.27. We have highlighted the whole period (called *long*) and a period ranging from May 15 to December 4 2022, called *short*. We chose the latter interval to better assess the model under heavily volatile market conditions impacted by two exogenous events. The first of these occurred in June 2022: the TerraUSD-Classic (USTC) lost its Dollar peg, wiping out about \$500 billion of market capitalization (see [?] for a detailed description of the event). The second crash occurred in November 2022 and involved a popular cryptocurrency exchange: FTX. This constitutes a good example of an exogenous-force-driven crash; [?] concludes that the major fault lay with the management and not in the crypto environment.

In order to acquire a complete overview of the trading, we also reported the number of trades and the maximum return per trade obtained in the period in Table 4.

As expected, the Dummy model returns slowly decrease to zero, yielding negative ROIs, reported in the figures in logarithmic scale. The long backtest

Period	Coin	MLP Max Ret	Dummy Max Ret	MLP trs	Dummy trs
long	ALGO	0.54	-0.99	376	643
short	ALGO	0.18	-0.42	36	110
long	BTC	0.29	-0.77	246	893
short	BTC	0.16	-0.44	28	106
long	ETH	0.76	-0.91	379	930
short	ETH	0.26	-0.41	37	111

Table 4: MLP and dummy model backtesting number of transactions and max return

shown in Figures 4a, 5a and 6a depicts a curve that is typical of a trading strategy with a statistical edge over a random one: a slow but stable increase in the return.

The short backtests in Figures 4b, 5b and 6b during the crashes expose the behavior expected from a strategy, mostly based on lagged indicators. In the next section will illustrate how lagged indicators determine most of the output of the MLP classifier. The initial Terra-Luna crash is smoothed by the MLP model in comparison to the Dummy model. The same behavior can be inferred from the FTX crash, near the end of the charts.

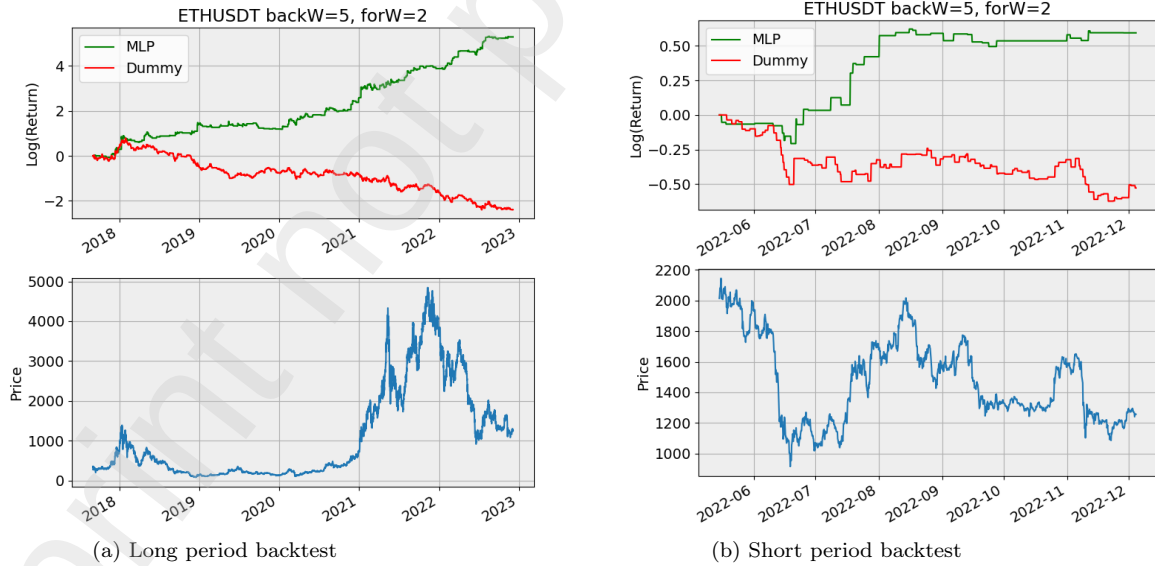


Figure 4: ETHUSDT backtest

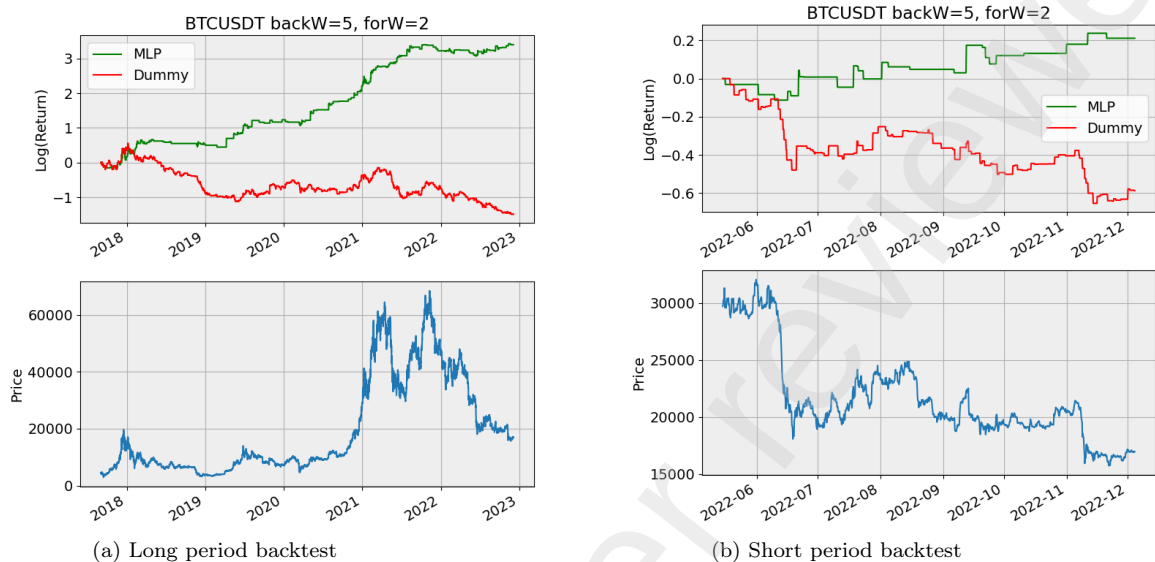


Figure 5: BTCUSDT backtest

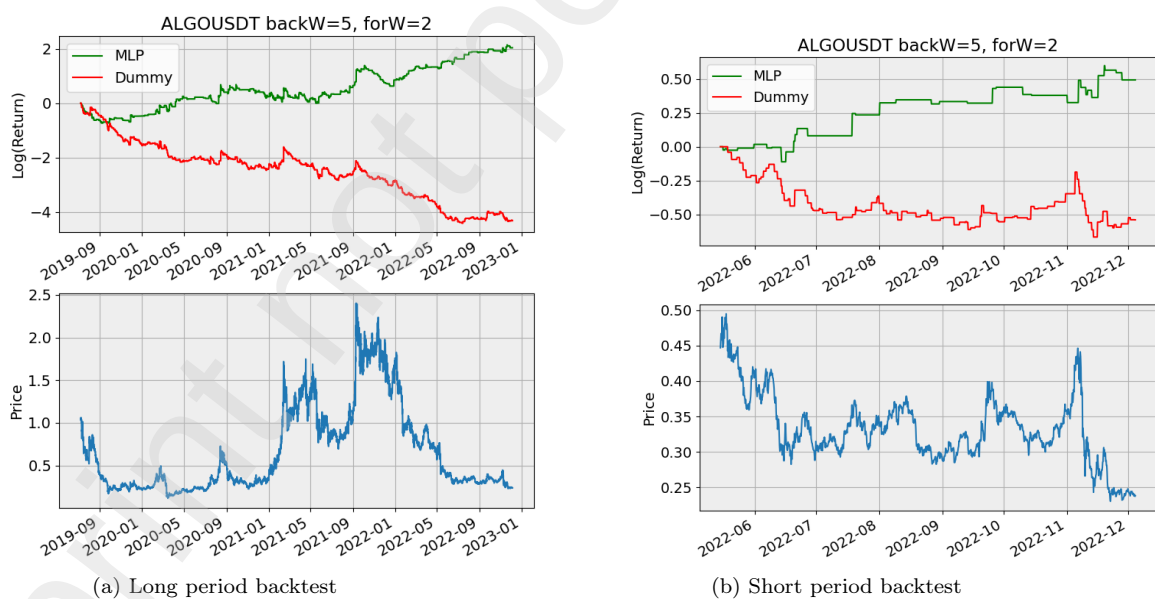


Figure 6: ALGOUSDT backtest

As seen in the short period backtests, our trading system performs very well against down trends. In the period ranging from February 15 to March

16, 2020 the [?] backtest for Bitcoin registered a loss of 28% (without commission fees), whereas our system exhibits a loss of 4.4% with commission fees applied.

6. Feature importance

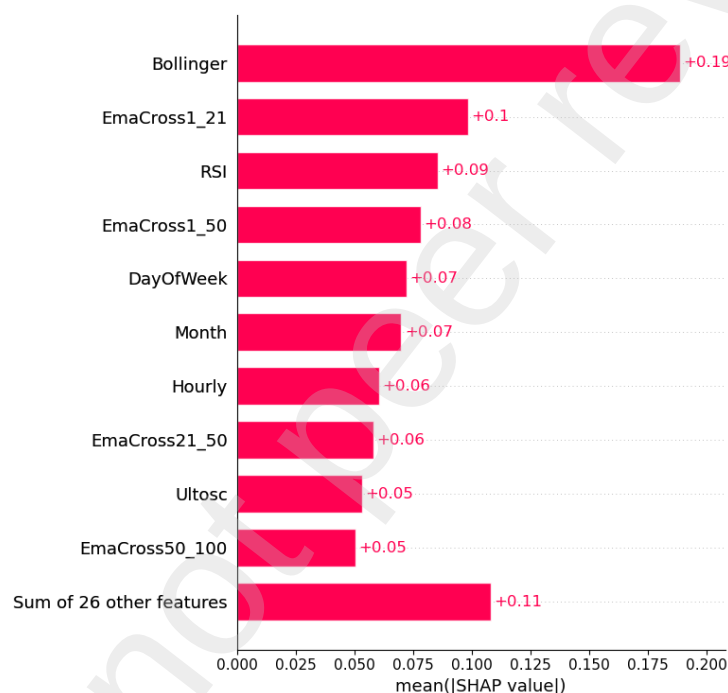


Figure 7: Top-10 feature mean contributions.

The field of interpretability/explainability in ML pertains to the comprehension of a model's behavior in predicting outcomes by mapping input to output. It is a continuously evolving area of study that aims to enhance the transparency of ML models [?]. Despite some exceptions (e.g. linear models, decision trees, random forests), where explanation of the model's behavior is accessible in terms of how much a feature determines the *global* model behavior, [?], MLP models have no direct way for their interpretation and are often treated as a black box oracle. Global model behavior provides a general understanding of how the model works: which features are most

important, and how they interact with each other. Global explanations are usually computed based on the analysis of multiple instances and are more suitable for generalizing the behavior of the model.

SHAP (SHapely Additive exPlanations), [?], is a game theoretic approach to explain the output of any black-box ML model. SHAP computes a value for the feature importance attribution, which is an implementation of the popular Shapley Values [?]. It is used to solve an attribution problem, by distributing the prediction of a model for a specific input to its base set of features and showing how influential a feature is in making a decision. Clearly, this approach provides a local explanation for each specific input of the model. In the implementation, we encoded the label **Buy** with -1, **Hold** with 0 and **Sell** with 1. SHAP values are in the range of the classifier output, and thus the values are in the $[-1,1]$ range although they may sometimes assume values slightly outside the range boundaries.

Overall feature importance can be inferred by computing the mean of SHAP absolute values for each feature, as shown in Figure 7.

The chart in Figure 8 shows the top-10 contributions of each feature to each prediction. More precisely this chart can be used to spot correlations between feature values and SHAP values using colors. For example, the Bollinger feature has a clear correlation with the output of the classifier, increasing the Bollinger value (on the vertical axis) and the output steers from **Sell** to **Buy** (on the horizontal axis). It is counterintuitive since in the trading literature a higher value of the Bollinger indicator means trend inversion. However, the model learned Bollinger values as a trend continuation feature. EmaCross1_21 and RSI, display a similar behavior. EmaCross21_50, instead, shows an inverse correlation with its SHAP value. The remaining features do not exhibit a behavior that can be correlated with SHAP values.

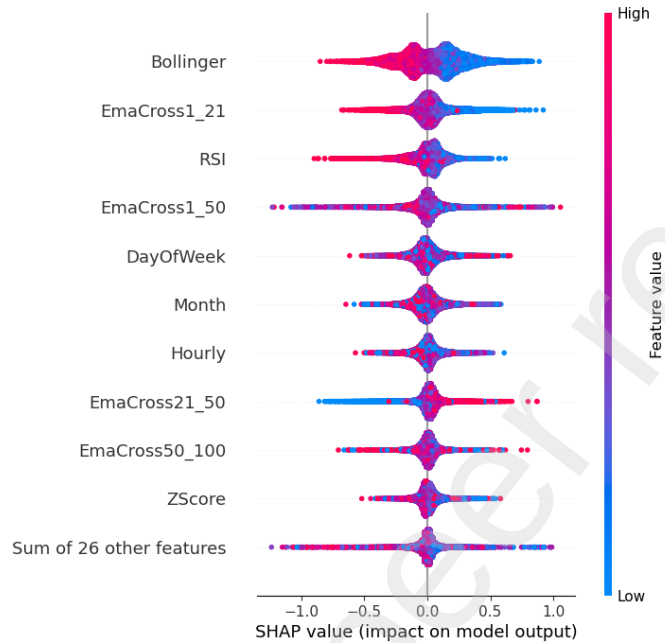


Figure 8: Feature impact on model output

As a local explanation, the SHAP values may be used to ask how a specific input can be explained in terms of feature values, or in other words, how a specific feature value steers the output of the classifier into the prediction. Waterfall plots achieve this by charting the contribution of each feature to the prediction. The chart in Figure 9 shows the SHAP values for the ten most important features. The vertical axis lists the features and their standardized values, while the horizontal axis reports the classifier output. The arrows show the extent to which every feature contributes to the prediction.

The impact of the remaining 26 features on the output is almost 0.

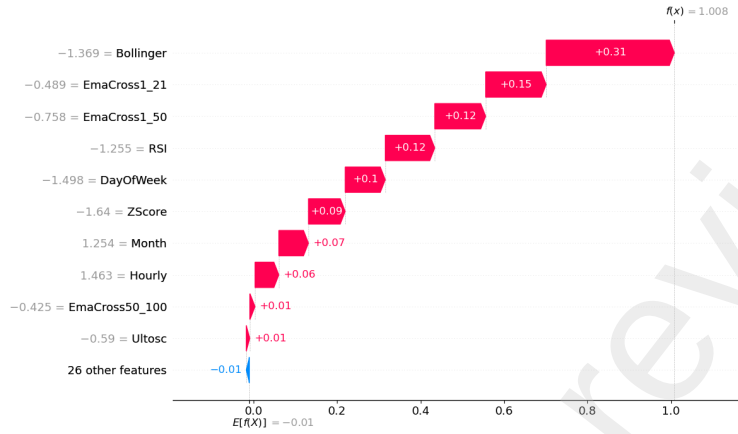


Figure 9: Prediction of an input vector in terms of top-10 features whose standardized values are on the left

In Figure 10, the output strays a few hundredths from the expected value 0 for the Hold label.

In conclusion, the investigation of the feature importance indicates that the top-10 most valuable features are comprised solely of technical indicators, moving average crossovers and temporal information. Candlestick patterns are seen to be relatively ineffective due to their infrequency or absence in some cases, and their ability to provide relevant information for determining the output of the classifier is limited.

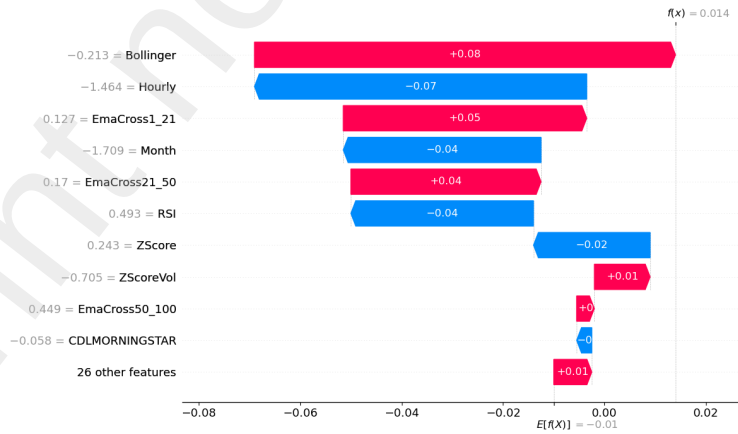


Figure 10: Prediction of an input vector in terms of top-10 features whose standardized values are on the left

7. Conclusion

The trading automation poses various challenges related to the feature extraction and labeling process, as well as the practical implementation of predictive models in real or simulated market settings. Prediction of the trend direction of a blockchain-based asset price presents multiple technical complexities due to poor regulation by authorities and market manipulations resulting in the pumping and dumping of selected assets [?], [?], [?] which leads to high volatility. Higher price fluctuations imply higher profit opportunities, but these profit opportunities are not easy to spot. The proposed approach reformulates the traditional forecasting model as a classification process. In particular, we have shown that a simpler approach supported by a massive dataset and smart labeling can achieve a comparable or better accuracy than more complex models. For example, a recent work [?] achieves a 56% accuracy on a binary labeled scheme. Under the assumption of a balanced dataset, this is only 6% over a random baseline. Whereas we have achieved a 66% accuracy, which is an excellent performance compared to a 33% baseline with a 3-classes problem.

We have developed a comprehensive pipeline for predicting short-term price trends and leveraging them to generate profitable trading strategies. Our findings reveal that the accuracy of price trend predictions is highly dependent on the use of a large dataset that enables the identification of patterns via technical analysis tools. We have shown that our trading system is profitable both in bull markets and in the more difficult bear/high volatile ones.

Furthermore, backtesting has demonstrated that a technical analysis approach can provide a clear statistical edge over random trading operations or a simple "buy and hold" approach.

Based on the analysis of feature importance, it has been determined that the top ten valuable features exclusively consist of technical indicators, moving average crossovers, and temporal information, with candlestick patterns showing a relatively low level of effectiveness.

These results lay the groundwork for future research endeavors that aim to augment the existing pipeline with additional information gleaned from multi-timeframe price action analysis. This approach can be accomplished by combining the pipeline's key components, such as the large dataset, labeling algorithm, best features and neural network, with novel information derived from a detailed analysis of price behavior across multiple timeframes.